Universidad Nacional de Córdoba – FaMAF
Ingeniería de Software I
2010

Django
Introducción

Diego Lis
Matías Bordese (updates v2010)

Objetivos de la charla

- ¿Qué es Django?
- ¿Por qué usar Django?
- Características generales
- ¿Cómo comenzar?
- ¿Dónde leer más?

Tener en cuenta para la arquitectura, diseño, implementación y testing del proyecto.

Aplicaciones Web

Problemas:

- Protocolo HTTP
- Mapeo objeto-relacional
- Generación de html
- Repetición de código / html
- Seguridad
- Eficiencia
- Interfaz de administración
- Etc.

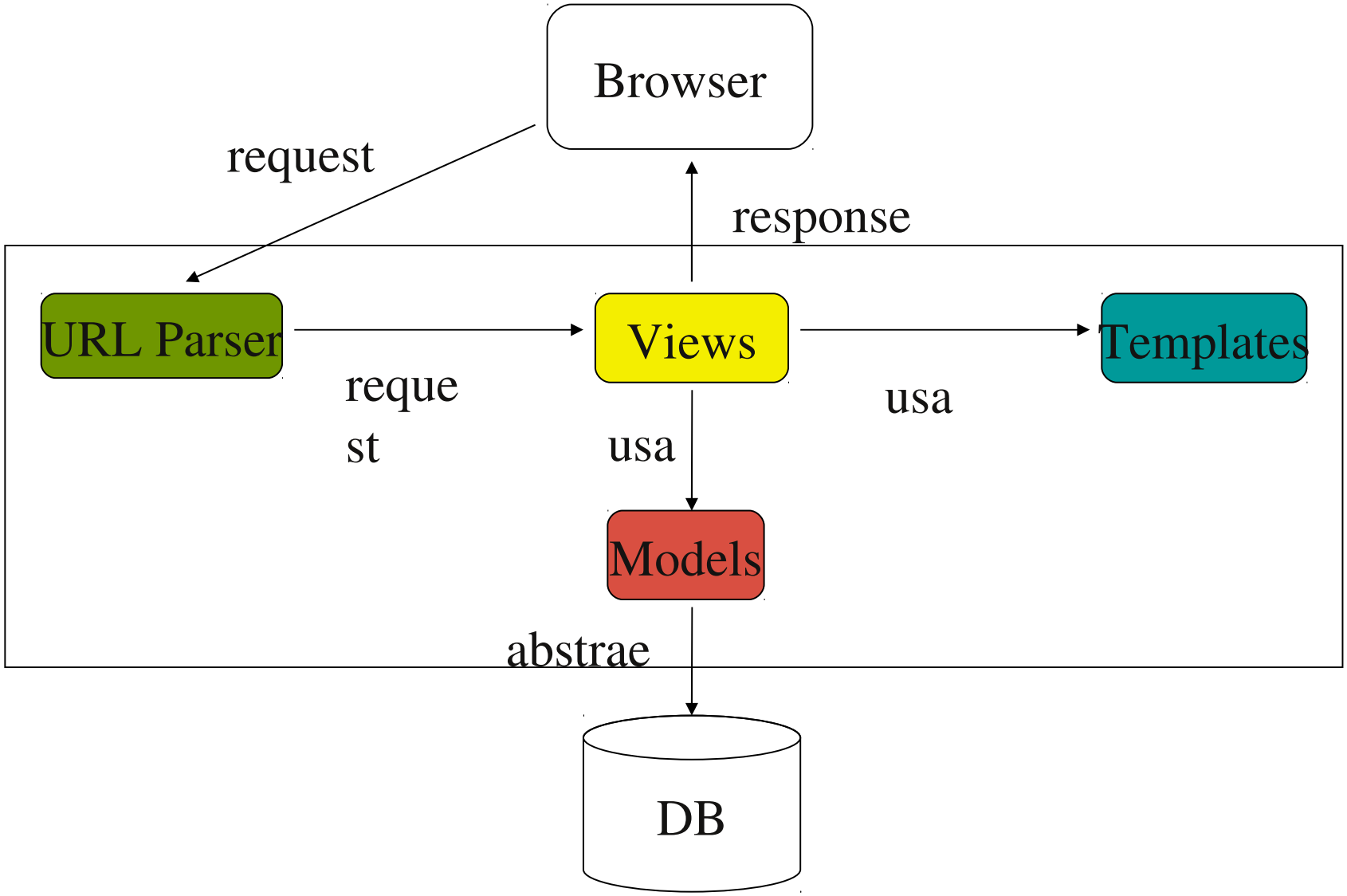
Framework

- Es una estructura de soporte definida en la cual otro proyecto de software puede ser organizado y desarrollado
- Facilita el desarrollo de software
- Permite pasar más tiempo identificando requerimientos de software que tratando con los tediosos detalles de bajo nivel

Django: Definición y principios

- Es un framework para el desarrollo de aplicaciones web, escrito en Python
- Se basa, entre otros, en los siguientes principios:
  - Bajo acoplamiento
  - Menos código
  - Agilizar el desarrollo
  - No repetirse a uno mismo (DRY)
  - Explícito es mejor que implícito

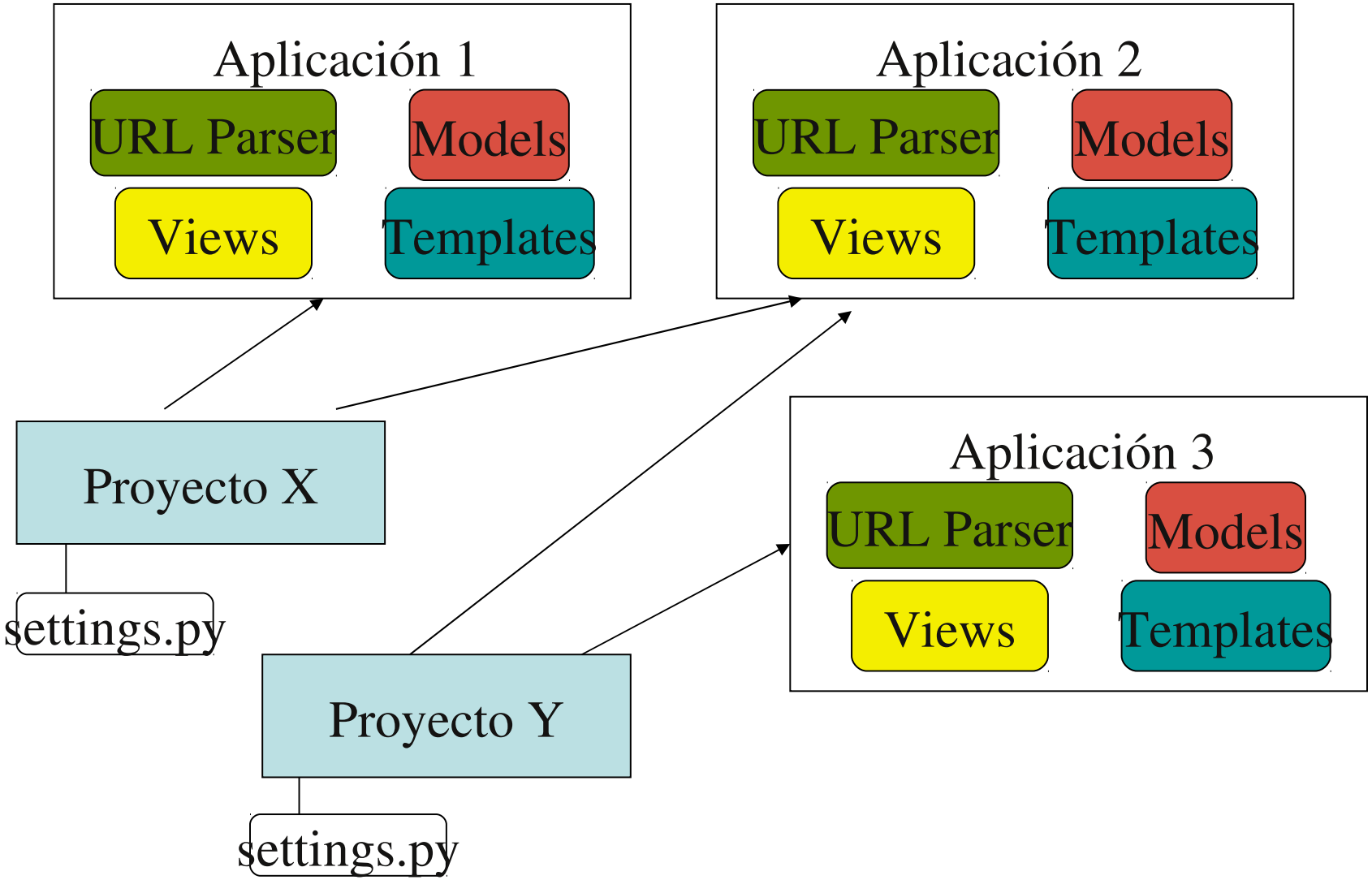
Esquema de un sistema en Django



Django: Revisión de la definición

Es un conjunto de **bibliotecas** que facilitan la creación de **models**, **views**, **templates** **y** **url parsers**.

Aplicaciones y Proyectos



Creando un proyecto desde cero…

- Instalar Python, Django
  http://docs.djangoproject.com/en/1.2/intro/install/
- Crear el proyecto

  > *django-admin.py startproject demo*
- Configurar *settings.py*
- Crear una aplicación

  > manage.py startapp libros

URL Parser

- Se utilizan **expresiones regulares** para parsear las URLs.
- Esto permite:
  - Diseño elegante y flexible de URLs. Por ejemplo, en vez de tener direcciones http://website.com/Reports?year=2008&month=1&day=12
  - podemos tener direcciones del estilo http://website.com/Reports/2008/01/12
  - Bajo acoplamiento entre las URLs y el código Python

URL Parser: código

http://localhost/libros/listar/ → listado de *todos* los libros
http://localhost/libros/235/ver/ → datos del libro *235*

```
urlpatterns = patterns('libros.views',
    (r'^libros/listar/$', 'mostrar_todos'),
    (r'^libros/(?P<id>\d+)/ver/$', 'mostrar_libro')
)
```

Parámetro

Número

De nombre *id*

Views

Un View es simplemente una **función** que toma como parámetro un request, y devuelve una respuesta para el navegador.

En el cuerpo de la función, se puede utilizar los models y templates.

Views: pseudocódigo

Información de request

def mostrar_todos (request):

obtener listado de todos los libros

devolver un html con dicho listado

Parámetro adicional

def mostrar_libro (request, id):

obtener los datos del libro id

devolver un html con dichos datos

Models

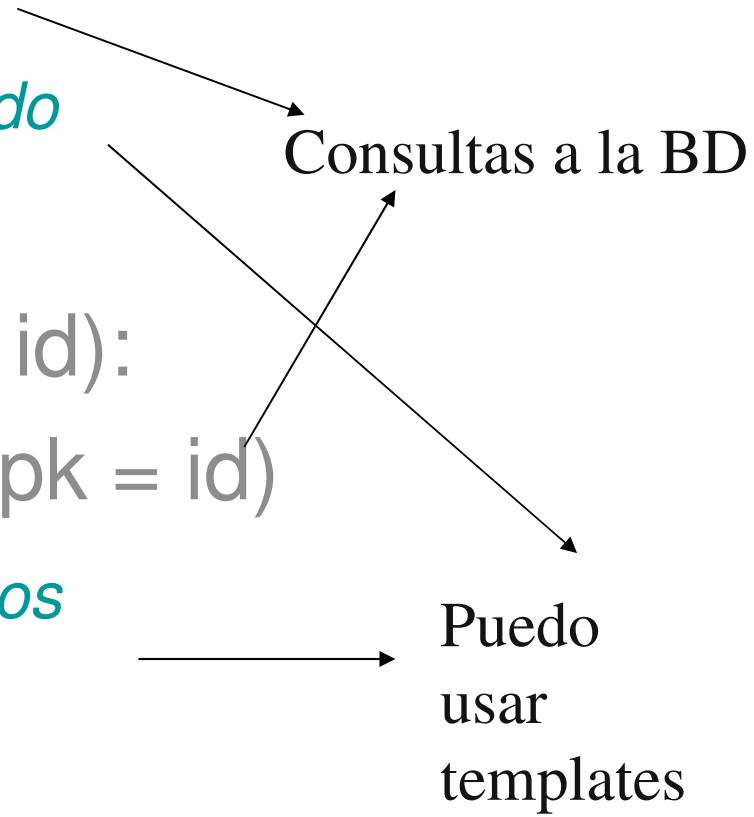
- Cada modelo es una **clase** que define un conjunto de atributos y un conjunto de métodos.
- El *ORM* de Django se encarga de crear la/s tabla/s necesarias en la base de datos, en forma **automática**.
- También permite realizar **consultas** a la base de datos totalmente vía objetos, sin utilizar *SQL*.

Models: código

```
from django.db import models

class Autor(models.Model):
    nombre = models.CharField(max_length=100)

class Libro(models.Model):
    titulo = models.CharField(max_length=100)
    autor = models.ForeignKey(Autor)
    fecha = models.DateField()

> ./manage.py syncdb
```

Views: pseudocódigo

```
def mostrar_todos (request):
    list = Libro.objects.all()
    devolver un html con dicho listado

def mostrar_libro (request, id):
    libro = Libro.objects.get(pk = id)
    devolver un html con dichos datos
```



Templates

- Permiten definir la capa de **presentación** de una aplicación
- Permite generar cualquier formato de texto plano, no sólo HTML
- Es simple de leer y de escribir
- No hace falta saber Python
- ¡Herencia de templates!
- Permite definir los propios comandos (tags y filters)

Templates: detalle.html

```
<html>
<body>
    <a href="{% url libros.views.mostrar_todos %}">Volver al
    listado</a>
            <p>Titulo: {{item.titulo}}</p>
            <p>Autor: {{item.autor.nombre}}</p>
            <p>Fecha: {{item.fecha}}</p>
    </body>
</html>
```

Views: código

```
def mostrar_libro (request, id):
    libro = Libro.objects.get(pk = id)
    return render_to_response("libros/detalles.html", {'item':
    libro})
```

Nombre del template

Parámetro para el template

Templates: index.html

```
<html><body>
    <table>
        <tr>
            <td>Titulo</td><td>Autor</td><td>Fecha</td>
        </tr>
    {% for item in list %}
            <tr>
    <td><a href="{% url libros.views.ver_detalle item.id %}">
            {{ item.titulo }}</a></td>
    </td>{{ item.autor.nombre }}</td>
    </td>{{ item.fecha }}</td>
           </tr>
    {% endfor %}
    </table>
</body></html>
```

Views: código

```
from django.shortcuts import render_to_response
from models import Libro, Autor

def mostrar_todos (request):
    lista = Libro.objects.all()
    return render_to_response("libros/index.html", {'list': lista})
```

Admin

- Interfaz CRUD (Create, Retrieve, Update y Delete)
- Suele ser la parte más aburrida de una aplicación
- Con Django, es automático

Admin: código

```
from django.contrib import admin
from models import Libro, Autor

admin.site.register(Libro)
admin.site.register(Autor)
```

Recursos

- http://www.djangoproject.com/
- http://www.djangobook.com/
- http://trac.usla.org.ar/django-book/
- http://www.wikipedia.org/
- http://groups.google.com/group/django-es
- #django-es en freenode (irc)

Preguntas

¿?